

Основные цели приложения

- **Уход от ручного ввода:** Раньше чтобы выдать права новому сотруднику, dba нужно было открыть терминал, вспомнить точную команду, без ошибок написать логин, пароль, имя базы и список таблиц. Если сотрудников много — это превращалось в бесконечную механическую работу, на которую тратились часы. В приложении всё это делается за пару кликов через простые формы и выпадающие списки.
- **Контроль увольнений в один клик:** Когда сотрудник увольнялся или переводился в другой отдел, dba приходилось вручную выискивать его учетные записи на всех серверах и поочередно писать команды отзыва прав (REVOKE). В панели достаточно найти пользователя и нажать одну кнопку — система сама мгновенно проверит все серверы и закроет ему доступ везде.
- **Освобождение времени для важных задач:** Приложение экономит до 30% времени dba. Приложение позволяет сконцентрироваться на выполнении других более приоритетных задач вместо ежедневной рутинной выдачи мелких доступов.
- **Оптимизация повседневных операций:** Перевод ручных низкоуровневых операций администрирования доступа в полностью автоматизированный декларативный контур, минимизирующий транзакционные и временные издержки персонала.
- **Унификация интерфейса:** Замена разрозненных консольных утилит и rsq1-сессий единой графической консолью с предустановленными шаблонами DDL/DCL-команд.
- **Работа без участия человека:** Представьте, что в компании на работу принимают нового программиста. Отдел кадров заводит его карточку в своей системе (например, 1С или корпоративный портал). Кадровая система сама, без звонков dba, «стучится» к нашей панели по API и говорит: *«Создай для нового сотрудника учетную запись на сервере баз данных»*. Приложение мгновенно выполняет команду и отправляет доступы на почту новичку. Всё происходит за секунду.
- **Автоматизация при деплое (CI/CD):** Когда разработчики обновляют приложение и создают новые таблицы, роботы-сборщики кода (например, GitLab) могут автоматически через API нашей панели выдавать нужные права для приложения на эти новые таблицы прямо в процессе обновления.
- **Моментальная реакция на угрозы:** Если система безопасности компании заметит, что аккаунт какого-то сотрудника взломали, она может мгновенно и автоматически через API дать панели команду: *«Заблокировать пользователя прямо сейчас»*. Приложение тут же перекроет доступ на всех серверах, предотвратив утечку данных, пока dba нет на месте.

- **Конец бесконечным переключениям:** Раньше чтобы выдать одно и то же право (например, на чтение таблиц) для одного сотрудника на пяти разных серверах, dba приходилось поочередно открывать пять консольных окон, вводить пять разных паролей и дублировать команды. Теперь он делает это в одной вкладке браузера.
- **Целостная картина перед глазами:** Из этого же окна dba видят общую картину: какие серверы сейчас активны, кто и к каким системам имеет доступы, а также сквозную хронологическую ленту логов. Информация больше не размазана по текстовым файлам на разных компьютерах.
- **Защита от опечаток в коде:** Исключение человеческого фактора при написании кода SQL (ошибки в синтаксисе). В случае человеческой ошибки база данных в лучшем случае выдаст ошибку, а в худшем — применит права не туда. Приложение само собирает команды по строгим цифровым правилам. Оператор просто выбирает параметры из готовых списков, а робот складывает их в код, где опечатки невозможны в принципе.
- **Контроль логики:** Приложение мгновенно проверяет действия оператора прямо в процессе ввода. Оно не позволит выдать права на несуществующую таблицу, отправить пустую форму или случайно заблокировать главного dba системы — встроенные блокираторы безопасности сразу пресекут такое действие.
- **Никаких простых паролей:** Человек по своей природе ленив или предсказуем. Создавая доступы вручную, пользователи часто используют простые, повторяющиеся или легко угадываемые пароли (например, Password2026, Admin123 или имя компании). Приложение полностью забирает эту задачу у человека. При создании новой роли робот мгновенно вычисляет случайный пароль, который состоит из длинного рандомного набора заглавных и строчных букв, цифр и спецсимволов. Подобрать его перебором невозможно.
- **Исключение утечек:** Пароль генерируется внутри оперативной памяти сервера. dba, который нажал кнопку создания пользователя, физически не видит этот пароль на экране и не может случайно скопировать его в Telegram, записать на стикер или переслать по незащищенному каналу.
- **Доставка прямо в руки сотруднику:** Система сама сохраняет пароль в зашифрованный контейнер, формирует временную ссылку и отправляет её на личную рабочую почту сотрудника. Пароль узнает только один человек — сам конечный пользователь, и только в тот момент, когда перейдет по одноразовой ссылке.

Особенности реализации и защиты приложения

- **Криптографический стандарт:** Шифрование учётных данных удалённых СУБД реализовано по спецификации **Fernet (симметричный алгоритм AES-128)** в режиме CBC с аутентификацией пакета через HMAC-SHA256.
- **Изоляция ключа (Volatile Memory):** 44-символьный Base64 MASTER_KEY считывается из переменных окружения Linux исключительно в ОЗУ процесса FastAPI. Запись ключа на постоянные носители (диски) заблокирована на уровне рантайма.
- **Атомарный сброс токенов (One-Time Tokens):** Одноразовые UUIDv4-идентификаторы сессий сотрудников хранятся в таблице one_time_tokens с жестким лимитом времени жизни expires_at.
- **Результат:** При первом GET-запросе к маршруту /api/view-password/ ядро бэкенда дешифрует пароль в оперативную память, выдаёт JSON-ответ клиенту и немедленно выполняет каскадный DDL-запрос DELETE FROM one_time_tokens WHERE token_id = %s. Это полностью ликвидирует вектор атаки, связанный с компрометацией архивов почтовых отправок или перехватом старых ссылок.
- **Защита забытого компьютера:** Если dba или менеджер вошел в приложение, отвлекся и ушел на обед, оставив компьютер открытым, ровно через 60 минут токен истечет. Доступ к приложению аннулируются.
- **Бесполезность украденных ключей:** Когда вы входите в систему, браузер получает временный токен. Даже если злоумышленник сможет как-то незаметно скопировать этот токен к себе на компьютер, он будет бесполезен ровно через час после его создания.
- **Принудительное выселение заблокированных:** Если в базе данных панели пользователя перевели в статус «Заблокирован», этот часовой таймер гарантирует, что его старый токен истечет, и он больше никогда не сможет подключиться к приложению.
- **Стандарт токенизации:** Аутентификация и контроль сессий операторов реализованы на базе стандарта **RFC 7519 (JSON Web Tokens — JWT)** с использованием криптографической подписи по алгоритму **HS256 (HMAC-SHA256)**.
- **Контур валидации времени (TTL):** При генерации токена в payload вшивается строгое timezone-aware свойство "exp" (Expiration Time), равное текущему UNIX-timestamp времени создания плюс ровно 3600 секунд.
- **Рантайм-блокировка бэкенда:** При каждом Ajax-запросе от фронтенда декоратор FastAPI Depends(RoleChecker(...)) извлекает заголовок Authorization: Bearer, проверяет валидность подписи через jwt.decode() и

сверяет время. Если `exp < current_timestamp`, рантайм мгновенно уничтожает транзакцию и возвращает статус HTTP 401 Unauthorized.

- **Результат:** Полная нейтрализация векторов атак, связанных с кражей сессионных кук, долгоживущим перехватом токенов (Token Replay Attacks) и несанкционированным доступом к брошенным рабочим местам (APM) администраторов.
- **Обман интерфейса:** Если обычный Аудитор (у которого есть доступ только к просмотру логов) окажется хитрым, откроет код страницы в браузере, вручную дорисует себе кнопку «Удалить сервер» и нажмет её — у него ничего не получится. Фронтенд отправит запрос на сервер, но там его встретит рольчекер.
- **Проверка на каждом шагу:** рольчекер не верит внешнему виду приложения. Он берет скрытый цифровой токен (JWT-токен) запроса, расшифровывает его на стороне сервера и смотрит и проверяет какой роли он соответствует.
- **Мгновенный разворот:** Происходит блокировка запроса и запись события в лог сервиса, о попытке входа под другой ролью.
- **Архитектурный шаблон:** Авторизация и разграничение прав доступа к API-маршрутам бэкенда реализованы на базе механизма **RBAC (Role-Based Access Control)** и внедрения зависимостей FastAPI через класс-декоратор **Depends(RoleChecker(...))**.
- **Серверная верификация роли:** При поступлении HTTP-запроса на эндпоинт (например, `/api/manage-privileges`), декоратор перехватчика принудительно извлекает и декодирует сессионный JWT-токен в ОЗУ с проверкой криптографической подписи секретным мастер-ключом.
- **Логическая блокировка безопасности:** Код извлекает строковое значение роли ("admin", "Security_Manager", "auditor") из доверенного payload сервера. Если текущая роль отсутствует в массиве разрешенных для данного маршрута ролей, рантайм атомарно прерывает исполнение функции и возвращает жесткий отказ **HTTP 403 Forbidden**.
- **ИБ-результат:** Полная ликвидация векторов атак класса **BOLA / IDOR (Broken Object Level Authorization)** и несанкционированного повышения привилегий (Privilege Escalation). Злоумышленник лишен возможности выполнить DDL/DCL команды в обход ограничений интерфейса за счет полного контроля бизнес-логики на стороне сервера.
- **Никаких скрытых команд:** Злоумышленник может попытаться схитрить и вписать в поле «Имя таблицы» конструкцию с вредоносным хвостом, например: `users; DROP DATABASE;`. Он надеется, что сервер послушно прочитает это как команду: *«Дай права на таблицу users, а потом сотри всю базу данных»*.
- **Кавычки-изоляторы:** Приложение никогда не склеивает текст от пользователя напрямую с командами базы данных. Она берет фразу `users; DROP DATABASE;`, упаковывает её в плотные защитные кавычки и говорит базе: *«Внимание, вот это длинное и странное слово — это просто имя таблицы, и ничего больше!»*.

- **Блокировка действий злоумышленника:** База данных PostgreSQL прочитает это как одно гигантское имя единой таблицы "users; DROP DATABASE;". Она просто поищет таблицу с таким именем, не найдет её и вернет обычную ошибку, а вредоносный код внутри точки с запятой никогда не сработает и не сотрет данные.
- **Механизм нейтрализации:** Защита от динамических инъекций кода на стыке приложения и СУБД реализована с помощью строгого разделения тела DDL-команды и пользовательских параметров по стандарту параметризации и безопасного экранирования идентификаторов.
- **компилятор DDL (psycorg.sql):** Бэкенд на Python полностью отказывается от уязвимого форматирования строк через f-строки или конкатенацию +. Для сборки динамических запросов применяется модуль psycorg.sql.
- **Принудительное токенирование:** Все входящие переменные (имена схем, таблиц, последовательностей и ролей) принудительно оборачиваются в класс `sql.Identifier(parameter)`. Компилятор драйвера автоматически преобразует их в безопасные строковые литералы PostgreSQL, экранируя кавычки, точки с запятой (;), комментарии (--, /*) и спецсимволы.
- **Результат:** Устранение критического вектора атак класса **A03:2021-Injection (по классификации OWASP Top-10)**. Любые попытки деструктивного внедрения SQL-кода через веб-интерфейс или внешние REST-интеграции приводят лишь к безопасным отказам удаленных СУБД из-за отсутствия объектов с аномальными именами.
- **Страховка от внезапного падения:** Если на сервере произошел сбой, отвалился интернет или бэкенд прислал ошибку, страница не зависнет с белым экраном и бесконечной загрузкой. Специальные ловушки в коде (try-catch) поймают этот сбой, аккуратно обработают его и выведут понятное предупреждение для оператора.
- **Защита от «кракозябр» и вирусов:** Если сервер пришлет ошибку, содержащую сломанные спецсимволы или скрытый вредоносный код, фронтенд не станет слепо выводить этот текст на экран. Специальный фильтр (санитизатор) мгновенно просканирует сообщение, хирургически вырежет из него все подозрительные символы или битую кодировку и покажет оператору идеально чистый текст.
- **Блокировка повторных кликов:** Чтобы оператор в панике не нажимал кнопку «Применить права» десять раз подряд во время медленного ответа сети (что могло бы вызвать перегрузку СУБД или создание дубликатов), фронтенд аппаратно блокирует кнопку (disabled) сразу после первого нажатия и разблокирует её только после полного завершения операции.
- **Контур изоляции исключений:** Все асинхронные Fetch-транзакции на стороне клиента (модули auth.js, rights.js) инкапсулированы внутри защитных блоков `try { ... } catch (err)`, что предотвращает аварийное

прерывание выполнения основного потока JavaScript при сетевых таймаутах или ответах класса HTTP 5xx.

- **Точная санитизация DOM-вывода:** Вывод динамического текста ошибок (data.detail или err.message) внутрь контейнеров alertDiv.innerHTML защищен регулярным выражением `/[^\x00-\x7Fа-яА-ЯёЁ\s\.,!\?-\]/g`. Этот фильтр очищает входящий поток от непечатаемых ASCII-символов, битых байтов кодировок (например, CP1251/UTF-8 легаси-дефектов) и потенциальных скриптовых инъекций.
- **Интерфейсный затвор (Debounce/Throttle Bypass Protection):** При инициализации события submit управляющие элементы UI (кнопки отправки DDL) принудительно переводятся в состояние disabled = true до момента разрешения (resolve) промиса от API. В блоке finally состояние сбрасывается обратно в false.
- **Результат:** Полная ликвидация векторов атак класса **Client-Side XSS (Cross-Site Scripting)**, защита интерфейса от парализации рантайма при нестабильном сетевом соединении и предотвращение гонки условий (Race Conditions) при отправке DCL-команд на удаленные кластеры СУБД.
- **Защита веб-сервера:** Nginx полностью скрывает от внешнего мира сам бэкенд на Python. Обычные пользователи и хакеры из интернета общаются только с Nginx по защищенному зашифрованному каналу (HTTPS на порту 443). Никто снаружи не может напрямую достучаться до внутренних частей приложения.
- **Двойной замок на документацию:** У панели есть встроенные автоподсказки и инструкции по коду для разработчиков (Swagger). Для обычных пользователей этот раздел полностью бесполезен, а для хакеров — это ценнейшая карта для взлома. Nginx ставит на этот раздел жесткую блокировку: если запрос к документации идет не от самого сервера (локально), а из внешней сети — веб-сервер мгновенно сбрасывает соединение и выдает ошибку.
- **Перевод кириллицы:** Nginx гарантирует, что все русские буквы в сообщениях об ошибках или статусах дойдут до браузера в идеальном виде. Он принудительно выставляет правильный стандарт кодировки, полностью ликвидируя появление ломающих интерфейс «кракозябр».
- **Архитектурный шаблон:** Внешний сетевой контур веб-панели изолирован с помощью развертывания высокопроизводительного веб-сервера **Nginx в режиме Reverse Proxy (обратного проксирования)**.
- **Терминация SSL/TLS и шифрование:** Веб-сервер принимает входящий трафик по порту **443 (HTTPS)**, обеспечивая шифрование сессий по строгим криптографическим протоколам **TLSv1.2 и TLSv1.3** с отключением уязвимых легаси-шифров (!MD5, !aNULL). Прямой HTTP-доступ по порту 80 аппаратно перенаправляется (Redirect 301) на защищенный HTTPS-контур.

- **Контур санитизации и кодировок:** Директива `charset utf-8`; на уровне сервера принудительно выравнивает метаданные HTTP-ответов, предотвращая дефекты интерпретации кириллических символов (`data.detail`) на стороне клиента.
- **Аппаратный ACL-блокировка для API-документации:** Доступ к интерактивным эндпоинтам Swagger (`/docs` и `/openapi.json`) жестко заблокирован на уровне конфигурации Nginx с помощью сетевых масок:


```
nginx
location /docs {
    allow 127.0.0.1;
    deny all;
    proxy_pass http://127.0.0.1:8000;
}
```
- **Результат:** Полное исключение рисков внешней разведки API-методов (API Reconnaissance). Прямой доступ к документации ядра из внешней сети физически невозможен (HTTP 403), а реальный IP-адрес бэкенда (`127.0.0.1:8000`) полностью скрыт за прокси-периметром Nginx, ликвидируя векторы атак прямого сканирования портов приложения.
- **Каскадная дистрибуция DCL/DDL:** Механизм веерной отправки пакетов, при котором бэкенд на Python принимает единый UI-Payload, валидирует его в ОЗУ и через неблокирующие потоки параллельно транслирует параметризованные SQL-команды на целевые эндпоинты кластеров СУБД.
- **Результат:** Полная ликвидация фрагментации контроля над учетными записями в масштабах предприятия. Исключаются ситуации «забытых» или скрытых избыточных прав на изолированных серверах, за счет обеспечения сквозной видимости матрицы доступов (Single Source of Truth) и централизованного логирования операций в системный аудит-журнал `audit_logs`.
- **Генерация случайных чисел (CSPRNG):** Создание временных паролей для учетных записей удаленных СУБД реализовано с помощью криптографически стойкого генератора псевдослучайных чисел на базе системного модуля `secrets` (библиотека Python), использующего аппаратную энтропию ядра Linux (`/dev/urandom`).
- **Политика сложности:** Рантайм-генератор формирует алфавитно-цифровую строку фиксированной длины (не менее 16 символов) с принудительным включением символов верхнего/нижнего регистров, цифровых рядов и спецсимволов, обеспечивая уровень энтропии, полностью исключающий атаки класса Brute-force.
- **Результат:** Устранение рисков, связанных с использованием слабых или скомпрометированных паролей (Weak Credentials). Гарантировано выполнение принципа «Нулевого доверия» (Zero Trust) на этапе заведения учетных записей: ни один из операторов или администраторов панели не имеет доступа к сгенерированному секрету, а сквозная

автоматизация доставки UUID-ссылок исключает человеческий фактор из цепочки передачи конфиденциальных данных.

Интерфейс интеграции (M2M): Реализация программного взаимодействия «машина-машина» на базе архитектурного стиля REST API с валидацией входящих структур данных через Pydantic-модели бэкенда.

- **Протокол авторизации:** Защита межпрограммного контура осуществляется с помощью передачи временных сессионных токенов по стандарту RFC 7519 (JWT HS256) в HTTP-заголовке Authorization: Bearer.
- **Результат:** Возможность бесшовной интеграции панели управления доступом с корпоративными экосистемами **IAM/IDM (Identity and Access Management)**, пайплайнами CI/CD и системами автоматизации реагирования на инциденты (SOAR). Исключается необходимость дублирования учетных записей и обеспечивается сквозная автоматизация жизненного цикла прав пользователей.